SEMTM0029: Financial Technology Group Project

Part A: Group Coursework Details (50%)

Academic Year: 2025/26

Guidelines:

1. **Deadline: 13:00 (UK time) on 7 May 2026.**

2. **Only one group member** should submit **a single PDF file** (name format: GroupNumber_ProjectName.pdf) to the submission point '**FTGP Group Report**' on behalf of the entire group.

3. This group coursework report should be formatted according to the Springer Lecture Notes in Computer Science (LNCS) conference-proceedings format, details of which (including templates for Microsoft Word and for LaTeX) are available here: https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines

4. The main body must be within a maximum length of 15 pages (a limit, not a target). The Reference and Appendix have no page limit.

5. The report should not simply serve as a list of achievements; it must be written in an academic writing style, featuring a logical progression and a coherent structure. It is essential to highlight your strengths and key points within the main text, rather than leaving it to the marker to scrutinize your code line by line to identify them.

6. This brief document contains important information regarding not only the assignment but also the relevant tasks you need to perform each week in group work weeks. Please read it carefully.

7. Other general guidelines see FTGP Group Summative Assignments Overview.

Project Execution:

In this project, you are required to build a distributed application (DApp) with a financial nature. **The user interface should be deployed on a public website, and the blockchain backend is deployed on the Sepolia testnet.** You are free to choose the exact topic your application will revolve around. You should follow the Scrum methodology.

Code Repository and Project Board:

At your first groupwork lab (week 21), your group should set up a PRIVATE repository on GitHub and name it as 'FTGP2526_GroupX_GroupName' to host your code. A JIRA project board (with same name format) is also required to be created to keep track of your work. For more details, please see week 21 material.

Your full-stack code, including front-end, back-end, and any testing code, must be included in both the appendix of the report (in text format) and uploaded to GitHub for the teaching staff to review. Please invite us to your GitHub repository (Staff email: zining.wang@bristol.ac.uk , charles.kind@bristol.ac.uk , and your team supervisor's email)**.**

Each member should contribute their work using commits in GitHub regularly as this can later be used as evidence of group work engagement and contribution. Github – Insights – Contributor will clearly show this.

The final deployed smart contract code should also be verified and published on Etherscan Sepolia (following this tutorial: https://chain.link/tutorials/how-to-verify-a-smart-contract-on-etherscan).

Contact Point (Lab):

During the final few teaching weeks of your project, you will have **contact points during the Monday lab session (except in week 24, you arrange a meeting with your supervisor at your convenience)**. There you will get the chance to discuss your progress and sprint report with one of the teaching staff. The teaching staff assigned to your team will ask you to talk through your group project progress and will ask each individual member to describe their work of the week. **This will be recorded and will be used as contribution evidence.** Attendance at these labs is compulsory for everyone, the teaching staff will make a record of attendance, and a valid reason must be given if you can't attend (illness, etc.). **Failure to attend to the scheduled contact points (labs) may result in a penalty. Continued absence without notice or explanation may place you at risk of failing the unit.**

Please note: Your team is expected to work autonomously on the DApp project. Staff will provide general guidance, facilitate discussions, and track your team's progress. However, they will not offer specific solutions to project-related technical questions. To ensure fair and equal

assessment, staff will not write, review, or debug your code, nor will they proofread your report or presentation.

Sprint Report:

When building your application, you need to provide **sprint reports** to the teaching staff on Blackboard (a template is available on blackboard). These must follow the scrum methodology and include:

1. A summary of your sprint review (could be supported by your project board screenshots) and sprint retrospective, that includes:
    a. Your progress, i.e. which tasks were complete and by who,
    b. Changes, i.e. tasks that have changed/not completed and why.
    c. Any retrospective reflections
2. A summary of your next sprint plan, that includes:
    a. The product owner and scrum master for the next sprint (you can assign the same team members in these roles for the duration of the project or switch for every sprint)
    b. Sprint Vision, i.e. what are your plans for the next sprint (week)
    c. Sprint Backlog, i.e. which items from the product backlog you plan on completing during the next sprint.

(For the first week, your report should start with your product vision and include a list of your product backlog.)

Your team's scrum master needs to submit your team's sprint report to Blackboard by Monday 17:00 p.m. (end of a lab), except in week 24, the report deadline is Thursday 17:00 pm. Detailed deadline dates are shown below. Failure to provide a report will result in a penalty on your team mark.

Date
16 March 2026
13 April 2026
20 April 2026
30 April 2026

Deadlines for Sprint Reports

Sprint reports are not mandatory during the non-teaching weeks (spring vacation or days in assessment week), and we might not review them in time. However, you are encouraged to continue writing them if your group is still working together and holding meetings. Doing so

may help you achieve a higher mark by demonstrating a stronger standard in the Project Management criterion (see the marking criteria details at the end of the document). All sprint reports can be included in the appendix of your group report.

Scrum Board:

Your group is required to create a JIRA scrum board (details in week 21 lab). As part of the final submission, provide clear screenshot of at least 'Summary', 'timeline', and 'backlog' page of your JIRA board.

Distribution of equity shares:

Discuss each member's contribution to the group. Use the GitHub commit history and the Scrum board as supporting evidence, and ensure that discussions are held with all team members.

Based on this equity share approach, the group can recognise members who contributed more to the project. The average of all weights must equal 1.0. For example:

Theo (0.95), John (0.99), Zining (0.95), Charlie (1.11).

Reflective Essay:

Around the end stage, after nearly completing all other project tasks, each group member should independently write an individual reflection essay of 500 words (±10%) and include it into the group report appendix.

To create a compelling reflective report, ensure you:

- Clearly state your name and provide a breakdown of your individual contributions to the project.
- Be critical and thorough, discussing what worked well and what didn't.
- Reflect on areas for future improvement and personal growth.

Bonus Points:

For additional bonus points, consider creating engaging content to pitch your application to users or investors. This could be (not limited to) a concise, TikTok-style video or an original short blog post (not copied from your group report) that introduces and explains your application. Take inspiration from examples like Tornado Cash's blog post (https://tornado-cash.medium.com/introducing-private-transactions-on-ethereum-now-42ee915babe0). Be creative and think outside the box!

Project Report Instructions:

Based on your DApp, you are required to produce a project report that contains at a minimum the following sections:

1. Introduction,
2. Background,
3. Design Documentation,
4. Project Execution
5. Conclusion,
6. Appendix.

The report should not simply serve as a list of achievements; it must be written in an academic writing style, featuring a logical progression and a coherent structure.

We highly recommend that you begin drafting this document concurrently with the development of the DApp during the teaching weeks, instead of waiting until the DApp development is complete to start writing.

Below, we have outlined the detailed requirements for each section:

<u>Introduction:</u>

In the Introduction, you are expected to introduce your DApp and explain its importance. Please keep this not overly technical and focus on uses/application. Here are a couple of potential questions, you should answer in this section:

- What exactly is your application? What can it do?
- Why is it important?
- Who is your target user group?

<u>Background:</u>

Here you need to describe the topic of your application in more detail, the area in which it resides and explain why it's needed, what are its use cases and why. You also need to briefly present other similar applications, potential competitors, or research conducted in this area. In addition, you need to think about the business side of things, do some financial research and talk about how such an application could be profitable (e.g. like a start-up). This section should include but not limited to the reading and citations of published papers, articles, and books as well as searching for other projects online (e.g. on GitHub, etc.). Remember to reference and back up your claims.

<u>Design Documentation:</u>

This section should include your system's design, describing its overall architecture and User Interface (UI) designs. Here are some ideas of what you might include in this section:

- Short introduction about design choices, to justify your team's approach to designing and building your application (see this link for further details),
- Step by Step explanation of how your DApp works (ideal usage), including all use cases,
- Use Case diagrams along with an overview of functionality and interactions,
- Sequence/Flow diagrams to highlight important and interesting cases of how the users interact with the application, along with a concise explanation,
- State/Data flow diagrams,
- User Interface section (Design of front-end, wireframe diagrams, etc.).

Project Execution:

This section shows how you worked to build your DApp and might include:

- Core code analysis, where you explain your implementation choices,
- Include any code/considerations that you believe warrants a detailed explanation, showcasing the quality and sophistication of your programming.
- A description of system functions and data structures, e.g. using UML diagrams,
- Your testing strategy, including what sort of testing methodology you have used,
- A showcase your final UI, e.g. using screenshots,
- A User Manual, where you present a guide to end-users on how to use your DApp.

Critical Evaluation:

In this section, you are requested to write a technical analysis of your DApp. Primarily, you could analyze the gas usage of your smart contracts, the security of your DApp, it's performance (Runtime Analysis) etc. You might also wish to compare your DApp with other similar applications, either traditional applications or other DApps.

Conclusion:

The conclusion section is a concise overview of your main findings, both theoretical and technical. You need to specify your achievements, any limitations, and potential future work.

Reference:

Ensure that all references are listed here and accurately cited within the main text.

Appendix:

The appendix has no page limit and must include the following items in the specified order:

- Your smart contract address(es) on Sepolia testnet. Link(s) to your smart contract(s) on Etherscan Sepolia testnet (Solidity Code should also be verified and published there).

- A link to your deployed website.
- Any deploy/test transaction hashes along with brief explanations (details included in the main text).
- A link to your project's GitHub repository (staff email should already have access).
- Distribution of equity shares within your group
- A screenshot of GitHub – Insights - Contributor.
- Required screenshot(s) for your JIRA project board.
- Each group member's individual reflection essay.
- All team sprint reports in chronological order, from the earliest to the most recent.
- The complete DApp Solidity smart contract code and testing codes, if any. (All codes should be in text format, we don't accept picture format; Use packages for clarity, see https://www.overleaf.com/learn/latex/Code_listing and solidity-latex-highlighting.)
- For front-end code, the complete version should be available on GitHub. In the appendix, only include the modified or relevant portions, such as page.js or other specific files. (Again, all codes should be in text format)
- Apart from these, you can include anything in the appendix, but keep in mind that the reader might only skim through this. There is no limit on the size of your appendix.

Marking Criteria:

Assessment Criteria	Fail	High Fail (near miss)	Pass	Merit	Distinction	High Distinction
1. Project Management (8/50)	No evidence of project management skills. The project lacks structure and planning. Roles, responsibilities, and timelines are undefined. There is no evidence of meetings, status updates, or risk management. Sprint report/JIRA project board is either non-existent or completely inadequate. The backlog either does not exist or is too high-level (lacking sufficient detail). GitHub repository is not accessible or only consists of commits by a single user, all completed around the deadline.	Minimal evidence of project management. There is a basic structure, but it is poorly defined and implemented. Roles and responsibilities are not clear, and timelines are unrealistic or not adhered to. Limited documentation in sprint reports, such as occasional meeting minutes or a basic project plan, may be present but lacks detail. Some records in project board and several commits of GitHub along the group phrase.	Basic project management practices are evident. There is a clear project structure with defined roles and responsibilities. There is adherence to some deadlines. Sprint reports are presented but could be more detailed. The project board is regularly updated, and there is evidence of adequate GitHub repository commits from multiple group members.	Good project management skills are displayed. The project is well structured with clear roles, responsibilities, and timelines, which are generally adhered to. Regular meetings demonstrate good communication and progress among team members. Comprehensive sprint reports and project board support the project's progress. One or two additional sprint reports after the teaching weeks.	Very good project management skills are evident. The project demonstrates excellent planning, structure, and adherence to timelines. Roles and responsibilities are clearly defined and efficiently executed. There is evidence of effective team collaboration, risk management, and regular status updates. The sprint reports and project board are thorough, well-organized, and include valuable updates after the teaching weeks. There is also evidence of good track of GitHub repository commits from multiple group members at some time points.	Exceptional project management skills. The project showcases outstanding planning, organization, and execution. Sprint reports and project board are exemplary and goes until the deadline, providing a clear and comprehensive record of the project's progress and management activities. There is evidence of good track of GitHub repository commits from all group members spanning the entire project timeline.
2. Style and communication (10/50)	Incoherent structure and presentation, making the arguments exceedingly difficult to follow. Simply list key accomplishments instead of presenting them in an academic essay format. Referencing is inaccurate and inconsistent. Communication skills show deep deficits.	The structure of the final project is sometimes difficult to follow and is not conducive to the development of reasoned arguments. Consequently, the communication of ideas is poor or inconsistent. Referencing is largely incorrect or incomplete. Figures and tables are pasted in from unusual outputs (e.g.: screen captures) with little attempt to format the results for professional presentation.	A reasonably good standard of communication with some appropriate use of academic language, terminology, and references. The structure allows the flow of arguments to be discerned, but improvements could have been made to support the progression of the analysis.	A good standard of presentation and writing with effective use of reference, the quality of writing facilitates the effective communication of ideas and understanding, but improvement could have been made to the structure and flow of ideas. Evidence of awareness of academic style in the presentation of results. The structure highlights key results, reporting findings, and includes notes below tables and figures that succinctly explain the result specifications, and the sources of data used.	Excellent presentation of ideas. Written in an academic style conducive to clear understanding of blockchain and financial ideas and their application. Excellent, logical structure and consistent writing throughout which was well supported by consistent and exhaustive referencing and research.	The presentation of ideas is exceptional, showcasing a profound understanding of the issues and their practical application. The writing is well-structured, consistent, and adheres to academic referencing standards, meeting the quality of an academic publication.

3. Motivation and Literature Review (6/50)	The topic is not sufficiently identified, nor is a rationale given to justify its financial application as the subject for an applied research project in decentralized finance. Insufficient coverage of literature and theory, with core areas, largely ignored. Wholly descriptive in style.	The topic is not clearly identified, nor is a clear rationale given to justify its selection as the subject for an applied research project in DeFi application. Limited coverage of literature and theory making few links to the research topic; descriptive in style.	The identification and justification for an applied research project in this area of DeFi is reasonably well presented. However, greater contextualization of its Financial or/and technical significance and the intellectual motivation for the project is needed. Reasonably good coverage of reading, but a somewhat limited range of resources utilized. The style tends more towards the descriptive but does include an element of critical evaluation.	Particularly good identification and justification of topic area with some originality. The topic has a clear DeFi focus, with its significance and intellectual motivation demonstrated effectively. Very good use of a range of resources demonstrating their relevance to the research topic. There is evidence of a critical evaluation of the reading.	Excellent identification and full justification of topic area, providing relevance of and intellectual motivation for choice. The topic has elements of originality and has significance beyond a student project. Evidence of a wide and appropriate selection and range of resources that are effectively used to demonstrate the relevance of the research topic. There is a critical evaluation of the materials.	Exceptional identification and full justification of topic area, providing relevance of, and intellectual motivation for topic choice. The research demonstrates originality and has a much wider significance and interest beyond a student project. An extensive and appropriate selection of a wide and varied range of resources. This is evaluated thoroughly and shows a mastery of relevant literature.
5. Quality of DApp Coding and Application of methods (18/50)	The submitted work demonstrates a fundamental lack of competency in both coding quality and methodological application. The code is unstructured, undocumented, and unreadable, with critical security vulnerabilities. The logic is highly inefficient, with no effort toward gas optimization or resource management. No testing or validation processes are evident. Methodologically, there is no engagement with lecture materials or relevant theories, and the chosen analysis approach is either missing, entirely inappropriate, or applied incorrectly.	The work approaches minimal standards but remains insufficient. The code shows minimal structure or documentation, with significant readability issues and unresolved security risks. Inefficiencies in logic, such as gas-heavy loops or redundant storage, are pervasive, and testing is either absent or superficial. Methodologically, there is superficial use of lecture materials, with an analysis method that is partially relevant but applied inconsistently or incorrectly.	The work meets baseline expectations with a functional but limited approach. The code has basic structure and partial documentation, though readability is inconsistent. Minor security oversights and inefficiencies persist, but core functionality is achieved. Testing is limited to basic unit tests for primary functions. Methodologically, appropriate data, theories, or models are used, and the analysis method is acceptable though it lacks depth.	The work reflects a solid understanding of coding and methodological practices. The code is readable and organized, with standard documentation and basic security practices. Efficiency improvements are attempted, and testing covers core functionality through unit and integration tests. Methodologically, there is good use of lecture concepts, theories, and models, with an appropriate analysis method clearly linked to literature. The work shows clarity in execution and critical engagement, though innovation and depth remain limited.	The work demonstrates advanced technical and analytical proficiency. The code is well-organized, modular, and thoroughly documented, with strong security practices and efficient logic. Testing and validation are systematic. Rigorous testing addresses edge cases and integrates unit/integration tests. Methodologically, sophisticated data, theories, and models are applied, with rigorous justification for the chosen analysis method. The work reflects a strong grasp of both technical and theoretical requirements.	The work exemplifies exceptions in coding quality and methodological innovation. The code is exceptionally clean, modular, and professionally documented, with robust security measures and highly efficient architecture. Innovative testing frameworks ensure reliability. Methodologically, the work integrates cutting-edge theory and models, with rigorous justification for method selection rooted in literature. The work showcases originality in both technical execution and theoretical application. This level represents a polished, publishable standard of work.
6. Analysis and reasoning (8/50)	Little attempt to draw reasoned findings and conclusions from the analysis. Very limited sense of argument or original interpretation.	Attempts have been made to identify findings and conclusions, but they are not clearly supported by the analysis. The work lacks coherence and does not create a logical argument, with frequent	Reasonably well-argued, but with scope for greater critical analysis beyond the merely descriptive. Findings and conclusions are reached but need further development.	Very well argued in drawing out findings and conclusions. There is some appropriate critical analysis and synthesis of research.	The excellently constructed argument, with a high level of cumulative logic supporting the analysis and conclusions. Evidence of wide-ranging critical analysis and	Exceptional level of argument with clarity of narrative that is outstanding. It demonstrates extensive and appropriate critical analysis and synthesis of research which identifies

		repetition and/or contradictions.			synthesis of research.	scope for further research.